
Contents

IO(Input-Output)	1
Tipi di istruzioni per l'I/O	1
Registri dei controller	2
Interrupt	2
Trap(interrupt sincroni)	2
DMA	3
Meccanismo di interruzione di RISC-V	3

IO(Input-Output)

I dispositivi di input convertono un segnale fisico in una sequenza di bit(ad esempio il tasto della tastiera, in una sequenza ASCII/Unicode, oppure un microfono ecc...)

I dispositivi di output svolgono il lavoro opposto, convertono una sequenza di bit in un segnale fisico.

Tutti i dispositivi I/O sono collegati ad un **controller**(ad esempio per la stampante controllerà il getto dell'inchiostro, i rulli,...), un dispositivo hardware che interconnette il dispositivo I/O con la CPU. Questi controller hanno dei registri che possono essere scritti dalla CPU ma sono separati dalla CPU. Saranno quindi collegati sul bus del sistema.

Il processore legge direttamente i registri del controller dal bus indirizzi e dati e li carica nei suoi registri. Questo viene chiamato **I/O mappato in memoria**.

Tipi di istruzioni per l'I/O

- I/O con busy waiting, il processore esegue il programma e quando deve elaborare dati da un dispositivo I/O, la CPU chiede al dispositivo I/O continuamente se ha dati disponibili. Questa tecnica spreca cicli di CPU se non ci sono dati disponibili. Esiste anche una versione simile detta polling, dove la CPU ciclicamente richiede i dati dal device I/O.
- I/O con interrupt, il controller quando riceve dati “avverte” il processore attraverso l’interrupt. Il segnale di interrupt va sul bus di controllo. Una volta arrivato questo segnale il processore potrà elaborare questi dati(ci sarà bisogno di un’unità di controllo più complessa che porti il program counter all’indirizzo del programma che deve elaborare i dati, il controller dell’interrupt). Il problema dell’interrupt è che per grosse quantità di dati la coda del controller non sarebbe abbastanza grande per contenere questi dati quindi il processore verrebbe interrotto troppe volte

-
- I/O in DMA(Direct Memory Access), il controller ha accesso direttamente alla RAM, in questo modo il processore non viene mai interrotto durante la scrittura. L'interrupt potrebbe comunque essere usato per segnalare la fine della scrittura

Registri dei controller

- Dati di ingresso e uscita. Sono i registri che contengono i dati in input e output
- Comandi, sono i comandi che il controller invia alla periferica
- Stato della periferica, indica in che stato si trova la periferica

Interrupt

Gli interrupt sono asincroni rispetto al ciclo fetch-decode-execute della CPU, il tutto è gestito dal controllore degli interrupt che è un programma che controlla gli interrupt della CPU, esegue i seguenti passi:

- Il controller attiva una linea di interrupt
- Quando la CPU è pronta manda un ack sul bus
- Il controller invia il vettore di interrupt
- La CPU lo riceve, salva i valori dei registri(compreso il program counter) e infine i registri vengono sovrascritti dal controllore degli interrupt e poi ripristinati
- Il vettore di interrupt serve per trovare l'indirizzo del PC a cui serve l'interrupt
- Si modifica il PC con questo valore e si evita che un'altro interrupt possa interrompere quello corrente
- Infine si ripristina il valore dei precedenti registri

Problema principale: Se un interrupt viene interrotto da un altro interrupt cosa succede? Il processore avrà dei protocolli per gestire gli interrupt, questo perchè rimandare un interrupt può essere impossibile quindi bisogna stabilire delle regole precise. Tenzialmente quando un interrupt viene ricevuto, gli altri si disattivano(anche se per alcuni dispositivi non è possibile).

Gli interrupt sono quindi gestiti in modo trasparente perchè il programma in esecuzione non deve accorgersi che è stato interrotto.

Trap(interrupt sincroni)

Esistono anche interrupt sincroni per segnalare errori da parte del processore per un pezzo di codice sbagliato. Questi interrupt possono richiedere anche routine del sistema operativo.

Questi tipi di errore sono ad esempio: Divisione per zero, overflow, indirizzi non allineati, ...

Ci sarà quindi un gestore che gestisce queste eccezioni e viene detto gestore delle trap.

Esistono anche trap asincrone generate da dispositivi I/O.

Gestione dell'interrupt L'interrupt può essere gestito in due modi:

- Senza trap, chi ha programmato l'ISA legge il bit dell'overflow e nel caso va ad una istruzione che gestisce l'overflow
- Con trap, la condizione di overflow rilevata via hardware genera una trap che modifica il PC ad una locazione che punta al gestore delle trap

DMA

Il DMA è un componente che scrive direttamente in RAM da un controller I/O. Questo avviene in maniera trasparente, la RAM non sa chi scrive se CPU o DMA. Entrambi condividono il bus e il DMA è impostato dal SO.

Meccanismo di interruzione di RISC-V

Quando in un programma avviene un'eccezione, il program counter viene modificato e viene eseguito il gestore delle eccezioni che cerca di capire il tipo di eccezione e chi l'ha generata.

Questo gestore è scritto da istruzioni ISA e da altre istruzioni supervisor che non fanno parte dell'ISA. La fine di questo gestore è segnalata da SRET(Supervisor Return). Le istruzioni supervisor usano i 4096 control status register(CSR) che possono essere letti/scritti solo in modalità supervisor. Questo gestore è controllato a livello kernel.

Per passare al gestore dell'eccezioni ci sono due modalità:

- Saltare al contenuto del registro STVEC che contiene l'indirizzo alla prima istruzione dell'Exception Handler
- Trovare l'exception handler adeguato nella memoria centrale, c'è una tabella con i vari exception handler in base alla causa.

Il gestore delle eccezioni(Exception Handler) svolge i seguenti passi:

- Salvare tutti i registri
- Andare in modalità kernel
- Disabilitare l'eccezioni perchè non ne può gestire altre.

-
- Salva l'indirizzo dove si verifica l'eccezione nel registro SEPC
 - Identifica la causa dell'eccezione
 - Salva la causa nel registro SCAUSE

I registri vengono salvati o nei registri shadow(cioè copie dei registri principali) oppure in memoria centrale.